Practical: Feature Engineering for Telecom Tower Failure Prediction Using Google Colab

1. Aim

To perform data preprocessing and feature engineering on a telecom tower failure dataset using Python and Google Colab.

The workflow is:

Raw Telecom Data
↓
Data Exploration
↓
Data Cleaning
↓
Feature Engineering
↓
Encoding
↓
Feature/Target Separation
↓
Processed Dataset
↓
Machine Learning Model

Today's practical ends at Processed Dataset.

3. Dataset

Your CSV should contain:

Tower_ID
Temperature_C
Battery_Voltage
Power_Consumption_W
Signal_Strength_Percent
Fan_Speed_RPM
Humidity_Percent
Traffic_Load
Tower_Age_Years

Failure_Within_48Hrs

Explain:

Features = information used to make prediction

Target = what we want to predict

Here:

Features → Temperature, Battery, Power, Humidity, etc.

Target → Failure_Within_48Hrs

4. Open Google Colab

Go to:

Google Colab

New Notebook

Rename it:

Telecom_Tower_Feature_Engineering

5. Import Libraries

Code Cell 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Explain

Pandas → works with tables/datasets.

NumPy → numerical calculations.

Matplotlib → visualization.

6. Upload Dataset

Code Cell 2

```
from google.colab import files

uploaded = files.upload()
```

Run this cell.

A file-selection window will appear.

Select:

telecom_tower_failure.csv

7. Load the Dataset

Code Cell 3

```
df = pd.read_csv("telecom_tower_failure.csv")

df.head()
```

Explain

pd.read_csv() reads our CSV file and stores it in a Pandas DataFrame.

df.head() displays the first five rows.

8. Check Dataset Size

Code Cell 4

```
df.shape
```

9. Check Column Names

Code Cell 5

df.columns

Tower_ID
Temperature_C
Battery_Voltage
Power_Consumption_W
Signal_Strength_Percent
Fan_Speed_RPM
Humidity_Percent
Traffic_Load
Tower_Age_Years
Failure_Within_48Hrs

---

10. Understand Data Types

Code Cell 6

df.info()

Explain:

For example:

Temperature_C → float
Battery_Voltage → float
Tower_Age_Years → integer
Failure_Within_48Hrs → integer.

---

11. Statistical Summary

Code Cell 7

df.describe()

This gives:

count

mean

standard deviation

minimum

maximum

quartiles

---

12. Check Missing Values

Code Cell 8

df.isnull().sum()

Explain:

> Missing data can cause problems during model training, so we need to identify and handle it.

---

13. Handle Missing Values

For numerical columns:

Code Cell 9

numeric_columns = df.select_dtypes(

```
    include=np.number
).columns

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())
```

Then:

```
df.isnull().sum()
```

Explain median

If values are:

40, 42, 45, 47, 100

The median is:

45

Median is often useful for filling missing numerical values because it is less affected by extreme values than the mean.

---

14. Feature Engineering

Now tell students:

> Feature engineering means creating new useful variables from existing variables.

We will create four features.

Feature 1: High Temperature

Code Cell 10

```
df["High_Temperature"] = np.where(
```

```
    df["Temperature_C"] > 60,
    1,
    0
)
```

Check:

```
df[["Temperature_C", "High_Temperature"]].head()
```

Explain

We convert:

Temperature > 60°C → 1
Temperature ≤ 60°C → 0

Example:

Temperature High_Temperature

42 0
55 0
67 1
72 1

This is a binary feature.

---

15. Feature 2: Low Battery

Code Cell 11

```
df["Low_Battery"] = np.where(
    df["Battery_Voltage"] < 45,
    1,
    0
)
```

Check:

```
df[["Battery_Voltage", "Low_Battery"]].head()
```

Explain:

Battery < 45V → 1
Battery ≥ 45V → 0

Again, this creates a binary feature.

---

16. Feature 3: Power-Traffic Ratio

Code Cell 12

```
df["Power_Traffic_Ratio"] = (
    df["Power_Consumption_W"] /
    (df["Traffic_Load"] + 0.01)
)
```

Check:

```
df[
    [
        "Power_Consumption_W",
        "Traffic_Load",
        "Power_Traffic_Ratio"
    ]
].head()
```

Explain

Power-Traffic Ratio
=
Power Consumption / Traffic Load

The 0.01 prevents division by zero.

This is called a ratio feature.

---

17. Feature 4: Tower Age Category

Code Cell 13

```
df["Tower_Age_Category"] = pd.cut(
    df["Tower_Age_Years"],
    bins=[0, 3, 7, np.inf],
    labels=["New", "Medium", "Old"]
)
```

Check:

```
df[
    ["Tower_Age_Years", "Tower_Age_Category"]
].head()
```

Explain:

0–3 years → New
3–7 years → Medium
>7 years → Old

This converts a numerical variable into a categorical feature.

---

18. One-Hot Encoding

Now explain an important ML concept.

Our column contains:

New
Medium
Old

A model shouldn't interpret:

New = 1
Medium = 2
Old = 3

as if "Old" is simply numerically greater than "Medium."

So we use one-hot encoding.

Code Cell 14

```
df = pd.get_dummies(
    df,
    columns=["Tower_Age_Category"],
    dtype=int
)
```

Check:

```
df.head()
```

Now we get:

Tower_Age_Category_New
Tower_Age_Category_Medium
Tower_Age_Category_Old

For example:

Age Category	New	Medium	Old
New	1	0	0
Medium	0	1	0
Old	0	0	1

---

19. Remove Tower_ID

Tower_ID is just an identifier.

Code Cell 15

```
df = df.drop(columns=["Tower_ID"])
```

20. Separate Features and Target

Code Cell 16

```
X = df.drop(
    columns=["Failure_Within_48Hrs"]
)

y = df["Failure_Within_48Hrs"]
```

Explain:

X = independent variables / input features

y = dependent variable / target

In our case:

X
↓
Temperature
Battery
Power
Humidity
Traffic
Fan Speed
Tower Age
High Temperature
Low Battery
Power-Traffic Ratio
...

y
↓
Failure_Within_48Hrs

---

21. Check X and y

Code Cell 17

```
print("Number of features:", X.shape[1])
print("Number of records:", X.shape[0])
print("Target size:", y.shape[0])
```

---

22. Check Target Distribution

Code Cell 18

```
print(y.value_counts())
```

You can also visualize it:

```
y.value_counts().plot(kind="bar")

plt.xlabel("Failure Within 48 Hours")
plt.ylabel("Number of Towers")
plt.title("Tower Failure Distribution")

plt.show()
```

Explain:

0 → No failure
1 → Failure

This is important because if almost all towers are 0, the dataset may be imbalanced.

---

23. Save the Feature-Engineered Dataset

Code Cell 19

```
df.to_csv(
    "telecom_tower_feature_engineered.csv",
    index=False
)
```

Check:

```
df.head()
```

---

24. Download the Processed Dataset

Code Cell 20

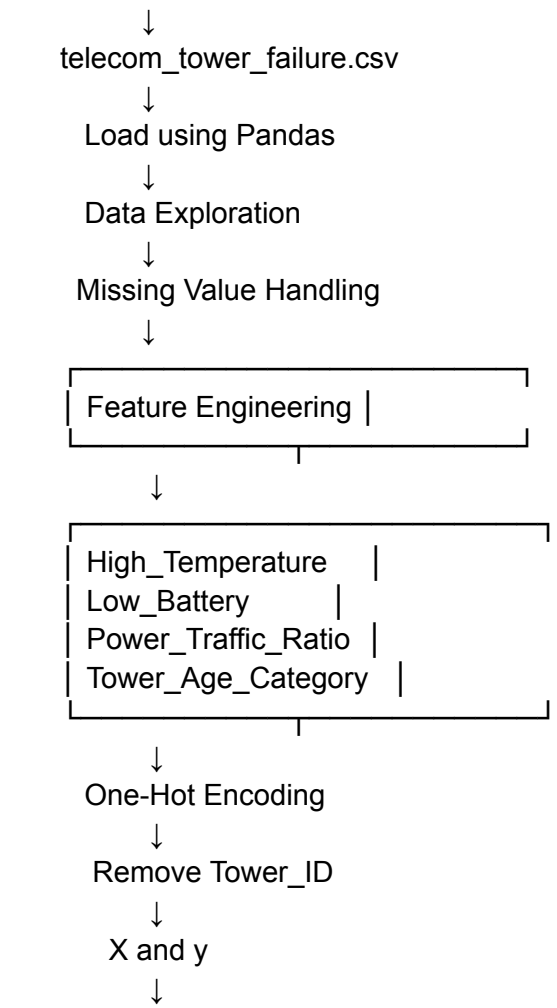
```
files.download(
    "telecom_tower_feature_engineered.csv"
)
```

The processed CSV will download to the student's computer.

---

25. Complete Workflow

Put this diagram in your practical record:



Feature-Engineered CSV
↓
Random Forest Training
NEXT PRACTICAL

26. One Combined Code Cell

After teaching the individual steps, you can give students the complete code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload dataset
uploaded = files.upload()

# Load dataset
df = pd.read_csv("telecom_tower_failure.csv")

# Display first five records
print("First five records:")
display(df.head())

# Dataset information
print("\nDataset shape:")
print(df.shape)

print("\nDataset information:")
df.info()

# Statistical summary
print("\nStatistical summary:")
display(df.describe())

# Check missing values
print("\nMissing values:")
print(df.isnull().sum())

# Handle missing numerical values
numeric_columns = df.select_dtypes(
    include=np.number
).columns
```

```
for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())

# Feature 1: High Temperature
df["High_Temperature"] = np.where(
    df["Temperature_C"] > 60,
    1,
    0
)

# Feature 2: Low Battery
df["Low_Battery"] = np.where(
    df["Battery_Voltage"] < 45,
    1,
    0
)

# Feature 3: Power-Traffic Ratio
df["Power_Traffic_Ratio"] = (
    df["Power_Consumption_W"] /
    (df["Traffic_Load"] + 0.01)
)

# Feature 4: Tower Age Category
df["Tower_Age_Category"] = pd.cut(
    df["Tower_Age_Years"],
    bins=[0, 3, 7, np.inf],
    labels=["New", "Medium", "Old"]
)

# One-hot encoding
df = pd.get_dummies(
    df,
    columns=["Tower_Age_Category"],
    dtype=int
)

# Remove Tower ID
df = df.drop(columns=["Tower_ID"])

# Separate features and target
X = df.drop(
    columns=["Failure_Within_48Hrs"]
)
```

```
y = df["Failure_Within_48Hrs"]

# Display final dataset
print("\nFeature-engineered dataset:")
display(df.head())

print("\nNumber of features:", X.shape[1])
print("Number of records:", X.shape[0])

# Target distribution
print("\nTarget distribution:")
print(y.value_counts())

# Save dataset
df.to_csv(
    "telecom_tower_feature_engineered.csv",
    index=False
)

# Download
files.download(
    "telecom_tower_feature_engineered.csv"
)

print("\nFeature engineering completed successfully!")
```

1. Why SageMaker/Colab? → Development environment for ML.

2. What is a feature? → Input information used by the model.

3. What is feature engineering? → Creating/transforming useful features from raw data.

4. Why encoding? → Convert categorical information into numerical representation.

5. Why X and y? → X contains inputs; y contains the value we want to predict.